

Uppsala University

Department of Materials Chemistry
Ångström Laboratory

EnGOAT

**The TuTraSt Algorithm–Based
Energy Geometry Analysis Toolkit**

Instruction Manual



Matevž Turk

Uppsala, 2025

Contents

1	Introduction	2
2	Software Requirements and Setup	2
3	Input Files	3
3.1	Input Parameter File	3
3.2	Cube File	4
4	Output Files	6
4.1	The <code>output.log</code> File	6
4.2	TuTraSt Data	11
4.3	Numpy Matrices	12
4.4	Tunnel Data	13
4.5	MSD Files	14
5	References	17
A	The TuTraSt Workflow	18
A.1	Construction of the Potential Energy Grid	19
A.2	TuTraSt analysis	20
A.3	Kinetic Monte Carlo Simulations	24

1 Introduction

EnGOAT (Energy GeOmetry Analysis Toolkit) is an open-source Python–Fortran package for analyzing potential energy surfaces of small species in crystalline solids. It is based on the TuTraSt algorithm [1], but extends its capabilities with additional functionality, improved performance, and a more user-friendly interface. The workflow takes as input a potential energy surface of a selected probe species within a crystal lattice (specific to a given probe–host pair) and provides energetic and geometric descriptors, as well as dynamic properties of the system such as diffusion coefficients. The method is applicable to a wide range of systems, from small molecules in porous crystalline materials to solid-state ion conductors [1–3].

This manual is divided into two parts: the main text and the appendix. The main text focuses on the practical use of the software, covering installation (Section 2), the structure of the input files (Section 3), and the structure of the outputs along with the meaning of the reported metrics (Section 4). The Appendix A provides a detailed description of the algorithm workflow, as well as the mathematical foundations and computation of the metrics presented in the output files.

2 Software Requirements and Setup

The EnGOAT software is written predominantly in Python, with certain performance-critical parts implemented in Fortran. To run the software successfully, both Python (version 3.10 or newer) and a Fortran compiler must be installed. Additionally, the software is intended for Linux environments, as the Python–Fortran wrappers are incompatible with Windows.

The following Python libraries are required:

- `numpy`
- `scipy`

Before running EnGOAT, the Fortran subroutines must be compiled. This can be accomplished using the built-in `f2py` compiler in NumPy. To compile all the subroutines automatically, simply run the **provided installation script** `install.py`.

Alternatively, each subroutine can be compiled manually by entering the following command in the terminal:

```
python3 -m numpy.f2py -c -m file_name file_name.f90
```

Here, `file_name` should be replaced with one of the following subroutine names:

- `find_neighbours`
- `initiate_cluster`
- `get_process_cross_vector`

Once all Fortran subroutines are compiled, EnGOAT is ready to use. To run the software, place the desired input files in the directory containing the code and execute the main script:

```
python3 EnGOAT.py
```

3 Input Files

EnGOAT requires two input files:

- *Input parameter file*: This file contains all the user-defined inputs required to run the code. It must always be named `input.param`.
- *Potential energy grid file*: This file contains the potential energy surface of the chosen probe within the selected crystal lattice, represented on a grid. The file uses the `.cube` format.

Both files must be placed in the directory in which `EnGOAT.py` is run. The structure of both files is discussed in detail below. The example input files presented in this section relate to the analysis of the Li-ion PES in lithium germanium phosphorus sulfide (LGPS) and can be found as a provided example in the directory `Examples/Li_ions_in_LGPS/Input_files` in the EnGOAT GitHub repository.

3.1 Input Parameter File

An example of the input parameter file `input.param` is shown in Listing 1. As noted above, this file *must* be named `input.param` for the software to recognize it.

Listing 1: The input parameter file `input.param`.

```
1 1          %Energy unit: 1=kJ/mol, 2=kcal/mol, 3=Ry, 4=eV, 5=Hartree
2 5          %Energy step in kJ/mol
3 35         %Energy cutoff in kJ/mol
4 0          %Dynamic E_step correction? 0=no, 1=yes, 2=stop at breakthrough
5 1          %Run kMC? 0=no, 1=yes
6 1          %Number of temperatures
7 1000
8 1000000    %Number of kMC steps to run
9 5          %Number of kMC simulations to run for averaging
10 6.941e-3  %Particle mass
11 LGPS.cube %Cube file name
```

The file is structured as follows:

- *Line 1*: Choice of the energy unit in which the potential energy grid is stored. An integer from 1 to 5 is accepted as input, corresponding to kJ/mol, kcal/mol, Ry, eV, and Hartree, respectively. The software reads this integer and multiplies the grid by the appropriate conversion factor to convert the energy values to kJ/mol.
- *Lines 2–3*: Define the energy step and the cutoff energy for the TuTraSt potential energy grid analysis (see Section A.2), both given in kJ/mol.
- *Line 4*: Specify whether a dynamic adjustment of the energy step should be applied in order to achieve the first breakthrough in the system between 10 and 20 energy steps (see Section A.2). Use 1 for yes and 0 for no. If the value 2 is entered, the algorithm will stop exploring the PES at first breakthrough (breakthrough energy = cutoff)

- *Line 5*: Specify whether kMC simulations should be performed to calculate the self-diffusion coefficient. If only the geometric descriptors are required, the kMC simulations are not necessary. Use 1 for yes and 0 for no.
- *Lines 6–7*: *Line 6* contains an integer N corresponding to the number of temperatures at which the kMC simulations are performed. The following N lines (in this example, N=1, so only *line 7*) contain the temperature values, given as floating-point numbers.
- *Lines 8–9*: Specify the number of simulation steps per kMC simulation and the number of independent kMC simulations performed to average the results (see Section A.3).
- *Line 10*: Specify the mass of the diffusing particle, given in kg/mol.
- *Line 11*: Specify the name of the `.cube` file from which the potential energy grid is read. The input is a string.

3.2 Cube File

An example of the cube file storing the potential energy grid data is shown in Listing 2. The name of this file is arbitrary, but it must be specified in the input parameter file (see Section 3.1) for the software to recognize it. The file is structured as follows:

- *Lines 1–2*: Comment lines of the cube file. These lines are ignored when reading the data.
- *Line 3*: The first integer specifies the number of atoms in the repeating unit. The following three values give the Cartesian coordinates of the origin of the volumetric data.
- *Lines 4–6*: The first integer on each line specifies the number of voxels (grid points) along each lattice direction. The following three values define the Cartesian components of the voxel basis vectors corresponding to the lattice vectors $\vec{a}$, $\vec{b}$, and $\vec{c}$, respectively. It should be noted that the Cartesian spatial coordinates in `.cube` files are given in units of *bohr*; the conversion to *Ångström* is performed internally by the software. If the coordinates are instead provided in *Ångström*, the resulting analysis will be incorrect.
- *Lines 7–17*: These lines specify the atom information: the atomic number (first value), the atomic charge (second value), and the Cartesian coordinates of each atom within the repeating unit (third, fourth, and fifth values, respectively). Although this information is not used in the TuTraSt analysis, these lines must be present. Importantly, the number of lines must match the number of atoms specified in *Line 3* to ensure correct program operation.
- *Lines 18–end*: The volumetric data encoding the potential energy grid. The data start at the origin and proceed through all values along the $\vec{a}$ direction, gradually filling the *ab* plane, and then filling successive layers along the $\vec{c}$ direction. The energy unit in which the volumetric data is given is arbitrary, but must be specified in the input parameter file (see Section 3.1).

Listing 2: The .cube file storing potential energy grid data of Li-ions in LGPS.

```
1 cube file - LGPS
2 -----
3 50 0.000000 0.000000 0.000000
4 44 0.377414 0.000000 0.000000
5 44 0.000000 0.377414 0.000000
6 64 0.000000 0.000000 0.373739
7 3 0.000000 16.419081 8.586925 22.764879
8 3 0.000000 16.016596 8.587440 10.197346
9 3 0.000000 8.478470 16.022358 13.130545
10 3 0.000000 8.242695 16.545138 1.211512
11 3 0.000000 4.314511 4.804394 3.769227
12 ... (40 more atoms)
13 16 0.000000 3.340988 7.947486 6.974460
14 16 0.000000 8.593069 3.337650 17.257294
15 16 0.000000 8.178130 13.162984 16.815480
16 16 0.000000 11.389805 0.264637 5.295828
17 16 0.000000 4.852469 0.517898 5.243373
18 89.946964
19 214.611112
20 214.611112
21 214.611112
22 214.611112
23 214.611112
24 214.611112
25 214.611112
26 214.611112
27 214.611112
28 214.611112
29 214.611112
30 214.611112
31 214.611112
32 214.611112
33 214.611112
34 214.611112
35 214.611112
36 214.611112
37 214.611112
38 214.611112
39 214.611112
40 214.611112
41 113.255355
42 62.893250
43 30.817063
44 12.097267
45 2.689335
46 ... (more volumetric data)
```

4 Output Files

EnGOAT generates several output files during execution. The primary output file, `output.log`, contains the bulk of the information and is written directly to the directory from which the code is run. In addition, supplementary outputs—such as matrices describing the positions of individual basins and transition states, minimum energy pathways, MSD files, and related data—are stored in dedicated subdirectories.

The structure of the main output file, `output.log`, as well as the contents and organization of the supplementary output directories, are described in detail below. The example output files presented in this section were generated using the example input files for Li-ions in LGPS described in the previous section, and can be found as examples in the directory `Examples/Li_ions_in_LGPS/Output_files` in the EnGOAT GitHub repository.

4.1 The `output.log` File

The `output.log` file records the progression of the simulation together with the key metrics evaluated during runtime. It is organized into multiple sections, each corresponding to a distinct stage of the EnGOAT analysis workflow. An example `output.log` file for Li-ions in LGPS is provided in Listing 3, followed by a detailed description of the contents and purpose of each section.

Listing 3: The `output.log` file for Li-ions in LGPS.

```
1 *****
2
3           E n G O A T
4
5 *****
6           The TuTraSt Algorithm Based Energy GeOmetry Analysis Toolkit
7
8           2026-03-20
9 *****
10
11 >>> INITIALIZATION <<<
12 -----
13 Input file:                input.param
14 Cube file:                 LGPS.cube
15 -----
16
17 >>> READING USER DEFINED INPUT <<<
18 Reading cube data...
19 Cube data read successfully.
20 -----
21 Number of atoms:           50
22 Origin (x,y,z):            ( 0.000, 0.000, 0.000)
23 -----
24 Grid points (Na,Nb,Nc):    ( 44, 44, 64)
25 a-axis vector:             ( 0.200, 0.000, 0.000)
26 b-axis vector:             ( 0.000, 0.200, 0.000)
```

```

27 c-axis vector:          (  0.000,    0.000,    0.198)
28 -----
29 Grid successfully loaded in 0.04 s
30
31 >>> POTENTIAL ENERGY GRID ANALYSIS <<<
32 Number of levels:          7
33 E step:                    5.0
34 E cutoff:                  35.0
35 -----
36 Level    Energy (kJ/mol)    Volume (pts)    Breakthrough    dt(s)    Total(s)
37 -----
38      1         5.00           74                /        0.03      0.04
39      2        10.00          400                /        0.08      0.11
40      3        15.00         1388                /        0.06      0.17
41      4        20.00         2976                /        0.11      0.28
42      5        25.00         5004          [0 0 1]        0.11      0.39
43      6        30.00         7226          [0 0 1]        0.17      0.56
44      7        35.00         9212          [1 1 1]        0.18      0.74
45 -----
46 Potential grid analyzed successfully in 0.74 s
47
48 >>> TUTRAST ANALYSIS <<<
49 Total volume:              72.6711 A^3      0.0743 %
50 Accessible volume:        65.1453 A^3      0.0666 %
51 Total surface area:       436.5707 A^2      0.4466 A^2/A^3
52 Accessible surface area:  391.5162 A^2      0.4005 A^2/A^3
53
54 1 tunnel systems were identified:
55 -----
56 Tunnel system #1
57 -----
58 Volume:                   65.1453 A^3      0.0666 %
59 Surface area:             391.5162 A^2      0.4005 A^2/A^3
60 Lowest energy point:      5.8672 kJ/mol
61 Dimensionality:           3
62
63 Breakthrough directions and energies:
64   Direction    | Highest E barrier (kJ/mol)
65   -----
66   [0 0 1]      | 14.6309
67   [1 0 1]      | 24.3924
68   [0 1 0]      | 24.3924
69   [1 1 0]      | 35.0000
70   [1 0 0]      | 35.0000
71   [1 1 1]      | 35.0000
72 -----
73 TuTraSt analysis performed successfully in 0.43 s
74
75 >>> RUNNING kMC SIMULATIONS (T = 1000.0 K) <<<
76 Number of runs:           5
77 Number of steps:          1000000

```

```

78
79 Tunnel system 1, weight = 0.7963145169643064
80 -----
81 Run          D_a (m^2/s)      D_b (m^2/s)      D_c (m^2/s)      dt (s)  total (s)
82 -----
83 1            4.365e-06        3.669e-06        1.288e-04        2.82    2.82
84 2            4.040e-06        3.707e-06        1.327e-04        2.74    5.56
85 3            4.606e-06        3.604e-06        1.331e-04        2.70    8.26
86 4            3.839e-06        3.852e-06        1.305e-04        2.70    10.95
87 5            3.596e-06        4.093e-06        1.307e-04        2.74    13.69
88 -----
89 Average self diffusion coefficient in a b c :
90 D_a = 4.090e-06+-3.61e-07  D_b = 3.785e-06+-1.74e-07  D_c = 1.312e-04+-1.57e-06
91 Average self diffusion coefficient in x y z :
92 D_x = 4.090e-06+-3.61e-07  D_y = 3.785e-06+-1.74e-07  D_z = 1.312e-04+-1.57e-06
93 Average Isotropic self diffusion coefficient:
94 D = 4.737e-05+-6.72e-07
95 -----
96 Completed kMC simulations for tunnel system 1 at T = 1000.0 K.
97 Total time elapsed: 13.71 s
98
99 -----
100 Joint diffusion coefficients across all tunnel systems at T = 1000.0 K:
101 In the a b c directions:
102 D_a = 3.257e-06+-2.87e-07  D_b = 3.014e-06+-1.39e-07  D_c = 1.044e-04+-1.25e-06
103 In the x y z directions:
104 D_x = 3.257e-06+-2.87e-07  D_y = 3.014e-06+-1.39e-07  D_z = 1.044e-04+-1.25e-06
105 Isotropic self diffusion coefficient:
106 D = 3.772e-05+-5.35e-07
107
108 >>> SUMMARY <<<
109 -----
110 Geometric descriptors
111 -----
112 Volume
113 Total:          72.6711 A^3      0.0743 %
114 Accessible:     65.1453 A^3      0.0666 %
115
116 Surface area
117 Total:          436.5707 A^2      0.4466 A^2/A^3
118 Accessible:     391.5162 A^2      0.4005 A^2/A^3
119 -----
120 Breakthrough energies
121 -----
122 Direction a:    level  7    E =    35.000 kJ/mol    dE =    24.392 kJ/mol
123 Direction b:    level  7    E =    35.000 kJ/mol    dE =    24.392 kJ/mol
124 Direction c:    level  5    E =    25.000 kJ/mol    dE =    14.631 kJ/mol
125 -----
126 Diffusion coefficients
127 -----
128 T (K)          D_a (m^2/s)+-sd      D_b (m^2/s)+-sd      D_c (m^2/s)+-sd

```

```

129 -----
130 1000.0      3.257e-06+-2.87e-07    3.014e-06+-1.39e-07    1.044e-04+-1.25e-06
131 -----
132
133 Total runtime:          14.93 s
134 Log file:              output.log
135 -----
136
137 Normal termination of EnGOAT.
138 *****

```

The sections of the `output.log` file are structured as follows:

1. *Initialization (lines 11–16)*: This section displays the names of the input files from which the simulation parameters and PES grid data were read.
2. *Reading User-Defined Input (lines 17–30)*: This section shows important parameters related to the repeating unit encoded in the `.cube` file, such as the number of atoms in the repeating unit (line 21), the origin of the Cartesian coordinate system (line 22), the number of grid points (voxels) along each unit cell vector direction (line 24), and the Cartesian coordinates of the unit cell vectors in Ångstrom (lines 25–27).
3. *Potential Energy Grid Analysis (lines 31–47)*: This section shows the progression of the flooding algorithm used to partition the PES grid into individual *basins* and *transition state surfaces* (see Section A.2). At the beginning, the number of energy levels, as well as the user-defined values for the energy step and the cutoff energy (in kJ/mol), are displayed. This is followed by a table summarizing the analysis at each energy level. The columns, in order, are: the energy level number; the corresponding energy value; the energy volume, i.e., the number of grid points at or below the current energy level; the unit cell vector direction of all breakthroughs recorded up to the current level ("," if no breakthroughs were recorded, or [a,b,c] if a breakthrough has been recorded, where a, b, and c are 1 if the repeating unit was breached in the given unit cell vector direction and 0 otherwise); the time spent analyzing the current energy level; and the total runtime.

If the *dynamic energy step correction* option is selected in the input parameter file (see Section 3.1), the energy levels are evaluated until the first breakthrough is recorded. If this occurs between levels 10 and 20, the analysis proceeds normally; otherwise, it is restarted with a modified value of the energy step. If *stop at first breakthrough* is selected, the analysis stops instead.

The final result of the potential energy grid analysis for the example case of Li ions in LGPS is illustrated in Figure 1. In this example, the flooding algorithm was carried out up to a cutoff energy of 35 kJ/mol with an energy step of 5 kJ/mol (see the input file in Listing 1). On the left, the LGPS structure is shown together with the Li-ion potential energy isosurface at 35 kJ/mol, visualized using VESTA software. The middle panel shows the individual basins identified by the flooding algorithm, while the right panel displays the transition-state surfaces between the basins.

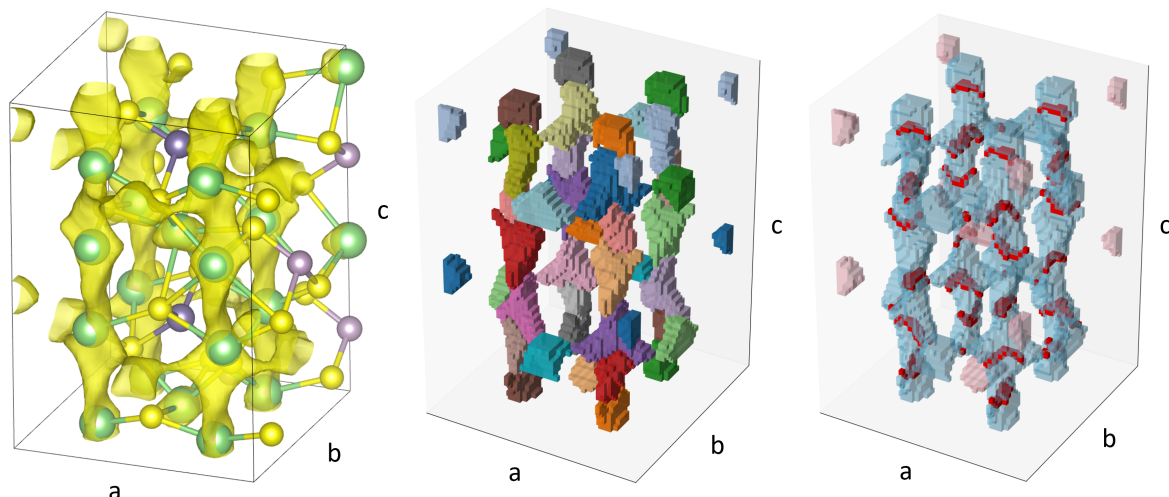


Figure 1: The PES landscape of Li-ions in LGPS. LGPS structure together with the Li-ion potential energy isosurface at 35 kJ/mol (left), Individual basins identified by the flooding algorithm (middle), and transition-state surfaces between the basins (right). On the right panel, the tunnel-forming basins (belonging to tunnel system 1, see the output file in Listing 3) are colored blue, and the non-tunnel-forming basins (basins that are not connected into a tunnel system) are colored pink.

4. *TuTraSt Analysis (lines 48–74):* This section presents the geometric descriptors resulting from the TuTraSt analysis of the PES grid, in which individual basins and transition states are grouped into *tunnel systems* (see Section A.2). First (lines 49–53), the *total* and *accessible* volume and surface area of the entire system are reported. The total volume and surface area are obtained by summing over all basins in the system, whereas the accessible quantities are summed only over basins that belong to tunnel systems (excluding the non-tunnel-forming basins – see Figure 1). The volume is reported in $\text{\AA}^3$ and as a percentage of the total grid volume, while the surface area is reported in $\text{\AA}^2$, as well as normalized by the total grid volume.

Subsequently, metrics describing each individual tunnel system are provided. In addition to the volume and surface area of a given tunnel system, the lowest-energy point within the tunnel system and the dimensionality of the tunnel system (defined as the number of linearly independent directions of percolating channels forming the tunnel system) are reported. Finally, a table listing all directions of percolating channels that constitute the tunnel system and their corresponding minimal highest energy barriers is provided for each tunnel system. Each direction is given by a vector $[a,b,c]$, where a , b , and c correspond to the number of unit cells traversed along the respective crystallographic directions. The minimal highest energy barrier that a particle must overcome to traverse a repeating unit along a given direction is obtained by applying the minimax algorithm, under periodic boundary conditions, to a graph representation of the tunnel system (see Section A.2). The identified diffusion paths corresponding to the minimal energy barriers are stored in the subdirectory `Tunnel_data/` and can be visualized using the `Minimum_E_Pathway_plotter.py` tool (for details, see Section 4.4).

5. *Running kMC Simulations (lines 75–107)*: This section presents the results of the kMC simulations performed for each tunnel system at each temperature specified by the user. At the beginning, the user-defined number of kMC runs (i.e., the number of independent kMC simulations used for averaging) and the number of kMC steps per simulation are reported. This is followed by one table for each tunnel system.

At the top of each table (line 79), the tunnel system identifier (ID) and its weight (the tunnel system's contribution to the total diffusion of the system, calculated from the Boltzmann probability of finding a particle in a given tunnel system (see Section A.3)) are reported. The table then lists the self-diffusion coefficients D_a , D_b , and D_c along the unit cell vector directions a , b , and c , respectively, together with the time spent on each run (Δt) and the cumulative runtime of the kMC simulations.

At the bottom of the table (line 138), the averaged self-diffusion coefficients and their corresponding standard deviations for the given tunnel system are reported along the a , b , and c directions, as well as along the Cartesian x , y , and z directions, together with the isotropic diffusion coefficient. The MSD files from all runs for all tunnel systems are stored in the subdirectory `MSD_files/` and can be visualized using the `MSD_plotter.py` tool (for details, see Section 4.5).

After all tunnel systems have been analyzed, the joint (weight-averaged) diffusion coefficients for the entire system at the specified temperature are reported (line 99–107).

6. *Summary (lines 108–138)*: This section presents a concise summary of the run. First, the geometric descriptors (volume and surface area) are listed, followed by a table showcasing the energy levels at which breakthroughs in each unit cell vector direction were first recorded, and the minimal highest energy barriers for traversing along these directions. At the end of the section is a table listing the self-diffusion coefficients along the crystallographic directions at all specified temperatures.

4.2 TuTraSt Data

After the TuTraSt analysis, information about basins, transition states, and tunnel systems is stored in a human-readable format in the directory named `TuTraSt_data`. This directory has the following structure:

```
TuTraSt_data/
|-- basin_data.dat
|-- TS_data.dat
|-- tunnel1_data.dat
|-- ... (additional tunnel system files if present)
```

- `basin_data.dat`: Contains a table summarizing the properties of each basin in the system. Each row corresponds to a single basin, with columns listing (in order): the basin ID; the ID of the tunnel system to which the basin belongs (/ if none); the position of the basin center—defined as the lowest-energy point in the basin—given as (a, b, c)

grid coordinates; the energy of this lowest-energy point (in kJ/mol); the basin volume (in Å³); and the Boltzmann-weighted basin volume at each user-specified temperature.

- Contains a table summarizing the properties of each transition-state surface in the system. Each row corresponds to a single transition state, with columns listing (in order): the transition-state ID; the energy of the lowest-energy point on the transition-state surface; and the pair of basins connected by the transition state.
- `tunnelX_data.dat` (where *X* is the ID of a given tunnel system): Contains detailed information about the corresponding tunnel system. At the beginning of the file, the *basin family*—i.e., the list of all basins comprising the tunnel system—is provided, followed by a table of all possible transition processes within the system.

Each row in the table corresponds to a single process, with columns listing (in order): the starting basin ID; the ending basin ID; the transition-state ID; the energy barrier of the process (defined as the difference between the lowest-energy points of the transition state and the starting basin); the grid coordinates (*a, b, c*) of the starting point (i.e., the center of the starting basin); the grid coordinates (*a, b, c*) of the end point (i.e., the center of the ending basin); and the process cross vector (*a, b, c*), which encodes periodic boundary condition (PBC) crossings. For each direction, this vector takes a value of 0 if no boundary is crossed, and ± 1 if the boundary is crossed in the positive or negative direction, respectively.

4.3 Numpy Matrices

The complete 3-dimensional matrices containing the locations of individual basins, transition states, and tunnel systems are stored in `.npy` format in the directory `NumPy_matrices`. This directory has the following structure:

```
numpy_matrices/  
|-- Basin_matrix.npy  
|-- Energy_matrix.npy  
|-- Level_matrix.npy  
|-- TS_matrix.npy  
|-- Tunnel_matrix.npy
```

- `Basin_matrix.npy`: A numpy matrix with the same number of points as the PES grid. Points belonging to a given basin contain the ID of that basin, while points not belonging to any basin are assigned a value of 0. Points that were identified as transition states are assigned the value of -1.
- `Energy_matrix.npy`: A numpy matrix with the same number of points as the PES grid. It contains the volumetric data from the `.cube` file, with the lowest energy shifted to 0 and all values above the cutoff set to the cutoff value.

- `Level_matrix.npy`: A numpy matrix with the same number of points as the PES grid. Each point holds the value of the energy level at which it was identified. Points in the PES grid with energy values above the energy cutoff are assigned the value *number of levels + 1*.
- `TS_matrix.npy`: A numpy matrix with the same number of points as the PES grid. Points belonging to a given transition state contain the ID of that transition state, while points not belonging to any transition state are assigned a value of 0.
- `Tunnel_matrix.npy`: A numpy matrix with the same number of points as the PES grid. Points belonging to a given tunnel system contain the ID of that tunnel system, while points not belonging to any tunnel system are assigned a value of 0.

4.4 Tunnel Data

The directory `Tunnel_data` contains detailed information about each identified tunnel system. The directory is structured as follows:

```
Tunnel_data/
|-- Tunnel1_hist.dat
|-- Tunnel1Txxxx.json
|-- min_E_path_d_tunnel_1.dat
|-- ... (min_E_path files for all breakthrough directions)
|-- ... (additional sets of files above for each identified tunnel system)
```

- `TunnelID_hist.dat` (where ID is the tunnel system ID): Contains the distribution of points within the tunnel system across energy levels, i.e., the number of points per energy level.
- `TunnelIDTxxxx.json` (where xxxx is the temperature; one file is created for each user-specified temperature): Contains the graph representation of the tunnel system at the given temperature, stored as a Python dictionary. The keys correspond to individual basins in the tunnel system. Each key maps to a tuple, where the first element is the Boltzmann probability of finding a particle within that basin, and the second element is a list of all transition processes originating from that basin. Each transition process is represented as a tuple with three elements: the end basin; the transition rate k , calculated using the Bennett–Chandler approach (see Section A.3); and the periodic boundary crossing information expressed as a crossing vector $[a, b, c]$.

Note: These files are created only if the user chooses to perform kMC simulations (see Section 3.1).

- `min_E_path_d_tunnel_ID.dat` (where d is the crystallographic direction a, b , or c): Contains a table listing the minimum-energy pathways along the crystallographic directions in a given tunnel system. Each row corresponds to a transition process along the pathway, with columns listing (in order): starting basin ID; ending basin ID; transition-state ID; lowest-energy point of the starting basin; lowest-energy point of the ending

basin; lowest-energy point of the transition state; fractional coordinate of the starting basin center along the d -direction; fractional coordinate of the ending basin center along the d -direction; and periodic boundary crossing information (0 if no boundary is crossed, and ± 1 if the boundary is crossed in the positive or negative direction, respectively).

The minimum-energy pathways in these files can be visualized with the provided `Minimum_E_Pathway_plotter.py` tool located in the `Tools/` directory in the EnGOAT GitHub repository. To run the script, place it in the folder from which EnGOAT was executed (next to the `Tunnel_data/` folder) and run it.

The script automatically detects all breakthrough directions for each tunnel system and generates two plots for each direction. The first plot shows the energy profile of the pathway, including basin and transition-state energies as well as basin positions along the d -direction. The second plot visualizes the tunnel system and highlights the basins involved in the corresponding pathway.

The two plots for each direction d will be saved as `.png` files entitled `Min_E_Path_d_tunnelID.png` and `Min_E_Path_a_tunnelID.Visualized.png`, respectively, and stored in `Min_E_path_plots/` directory. The minimum-energy paths for the example of Li-ions in LGPS are shown in Figure 2.

4.5 MSD Files

The directory `MSD_files` contains the mean-squared displacement (MSD) curves calculated during the kMC simulations, from which the self-diffusion coefficients are calculated. For each temperature, each tunnel system has up to seven files per run, corresponding to the three crystallographic directions a , b , and c , the three cartesian directions x , y , and z , and the full 3-dimensional (isotropic) MSD. The directory is structured as follows:

```
msd_files/
|-- msd_Txxxx_tunnel1_run1_directiona2.dat
|-- msd_Txxxx_tunnel1_run1_directionb2.dat
|-- msd_Txxxx_tunnel1_run1_directionc2.dat
|-- msd_Txxxx_tunnel1_run1_directionx2.dat
|-- msd_Txxxx_tunnel1_run1_directiony2.dat
|-- msd_Txxxx_tunnel1_run1_directionz2.dat
|-- msd_Txxxx_tunnel1_run1_isotropic.dat
|-- ... (additional sets of files above for each kMC run)
|-- ... (additional sets of files above for each identified tunnel system)
```

The MSD files can be visualized using the provided `MSD_plotter.py` tool, located in the `Tools/` directory of the EnGOAT GitHub repository. To use the script, place it in the directory where EnGOAT was executed (i.e., alongside the `MSD_files/` folder) and run it. The script automatically detects all tunnel systems and their corresponding breakthrough directions. For each tunnel system, it generates plots of all runs associated with a given direction at a specific

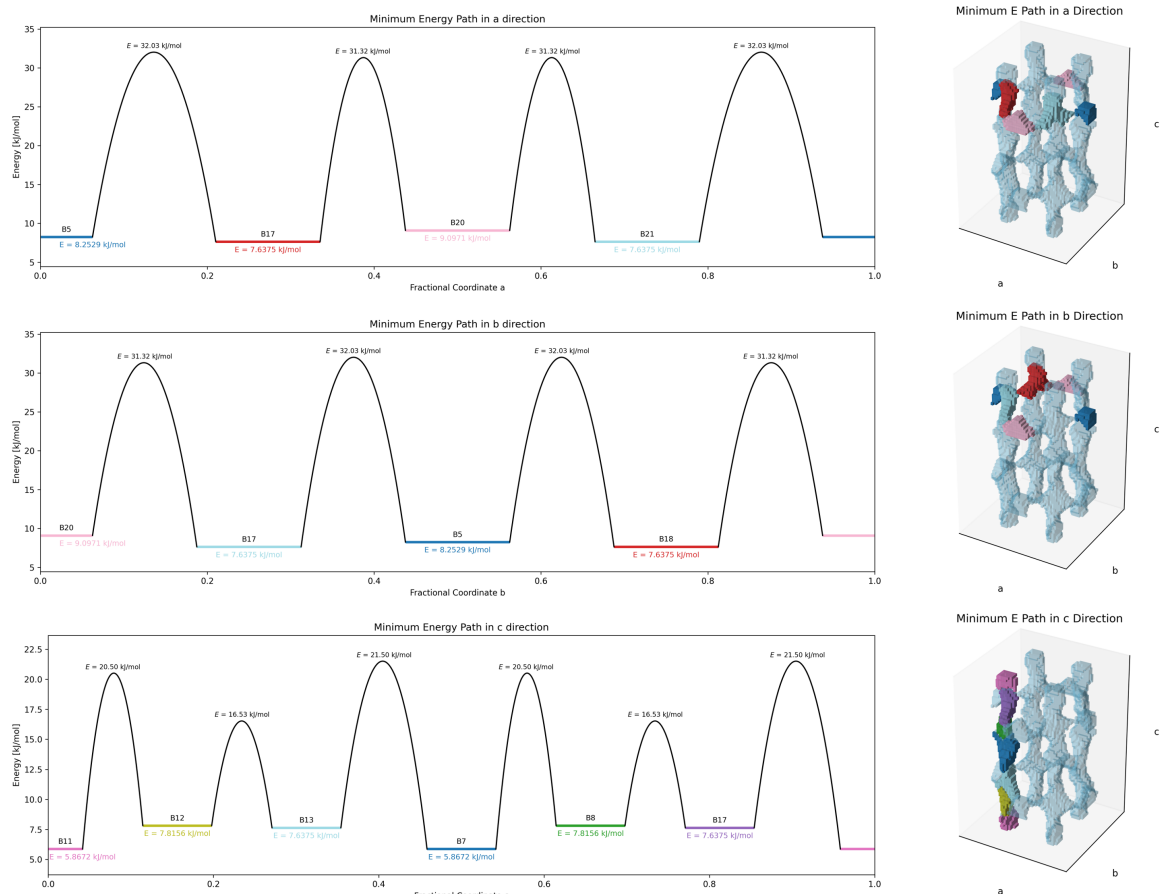


Figure 2: The minimum-energy pathways of Li-ions in LGPS along *a* (top), *b* (middle), and *c* (bottom) directions. In the energy profiles (left), individual basins are shown as colored horizontal lines. Each basin is labeled with its basin identifier (BID) above the line and its lowest energy value below. The position of the center of each line corresponds to the fractional coordinate of that basin along the given crystallographic direction. Transitions between basins are represented by black connecting lines, with the corresponding transition-state energies indicated above them. The images on the right show the structure of the given tunnel system (show in transparent blue), with the basins involved in the given minimum-energy pathway highlighted with the same colors as in the corresponding energy plots.

temperature, saving them as `msd_Txxxx.directiond2.png`. All generated plots are stored in the `MSD_plots/` directory. The MSD curves for the example of Li-ions in LGPS are shown in Figure 3.

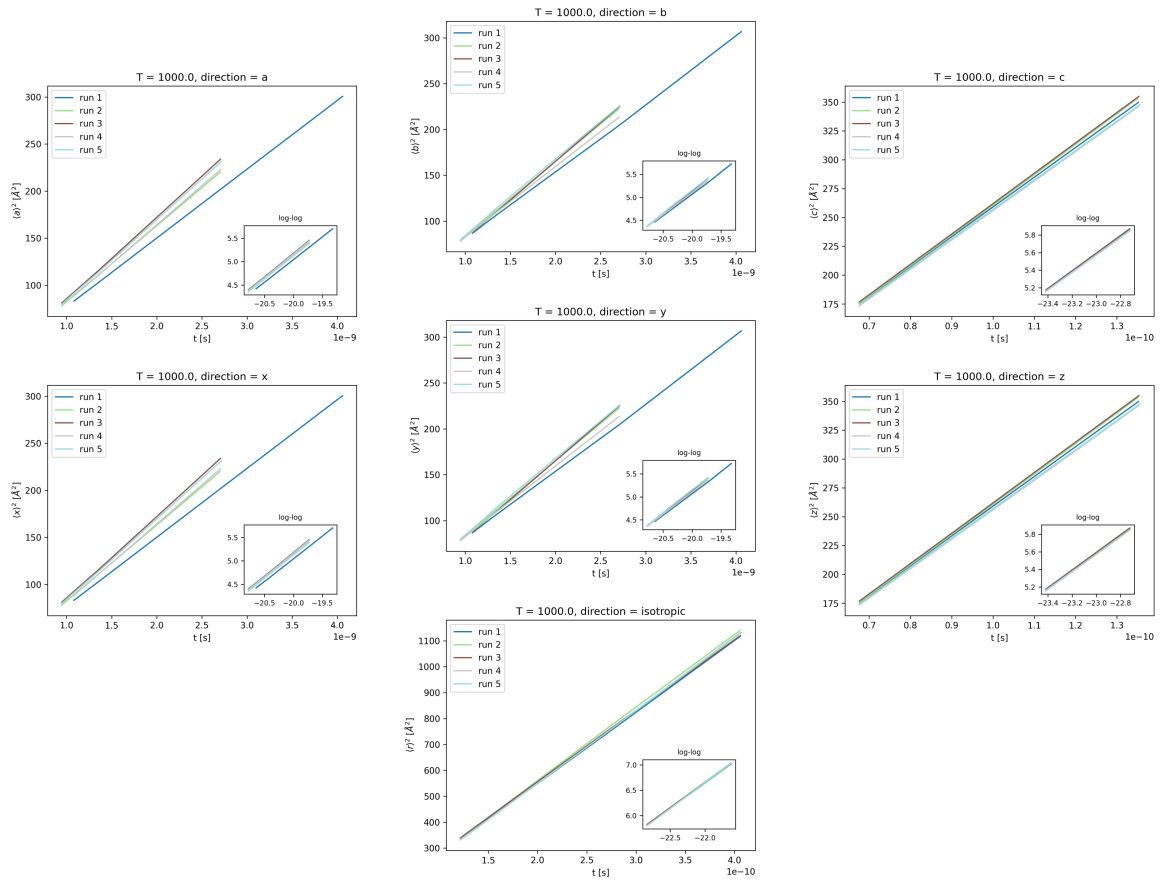


Figure 3: MSD curves for the example of Li-ions in LGPS along the crystallographic directions (top), cartesian directions (middle), and the isotropic MSD curves (bottom).

5 References

- (1) Mace, A.; Barthel, S.; Smit, B. Automated multiscale approach to predict self-diffusion from a potential energy field. *Journal of chemical theory and computation* **2019**, *15*, 2127–2141.
- (2) Gustafsson, H.; Kozdra, M.; Smit, B.; Barthel, S.; Mace, A. Predicting ion diffusion from the shape of potential energy landscapes. *Journal of Chemical Theory and Computation* **2023**, *20*, 18–29.
- (3) Schwarz, F.; Barthel, S.; Mace, A. Understanding Mobile Particles in Solid-State Materials: From the Perspective of Potential Energy Surfaces. *Chemistry of Materials* **2024**, *36*, 11359–11376.
- (4) Gustafsson, H.; Schwarz, F.; Smolders, T.; Barthel, S.; Mace, A. Computationally efficient DFT-based sampling of ion diffusion in crystalline solids. *Journal of Chemical Theory and Computation* **2025**, *21*, 8669–8682.
- (5) Frenkel, D.; Smit, B., *Understanding molecular simulation: from algorithms to applications*; Elsevier: 2023.
- (6) Bennett, C. H. Exact defect calculations in model substances. *Diffusion in Solids: Recent Developments.*, **1975**, 73–113.
- (7) Chandler, D. Statistical mechanics of isomerization dynamics in liquids and the transition state approximation. *The Journal of Chemical Physics* **1978**, *68*, 2959–2970.
- (8) Gillespie, D. T. A general method for numerically simulating the stochastic time evolution of coupled chemical reactions. *Journal of computational physics* **1976**, *22*, 403–434.

A The TuTraSt Workflow

In this section, the TuTraSt workflow [1] is explained in detail and illustrated using a simple example: a system containing a single diffusing particle moving through a two-dimensional, cell-centered tetragonal crystal lattice. The particle can occupy vacant basins located at the centers of clusters of four atoms, and can move between them via transition states situated between the two atoms that connect neighboring basins, as shown in Figure 1 (left). Such a crystal lattice can be described by a repeating unit, highlighted in Figure 1 (right). It is important to note that the choice of the repeating unit used in the TuTraSt workflow is not always straightforward and depends on the system in question [4], a point that will be further discussed in Section A.1.

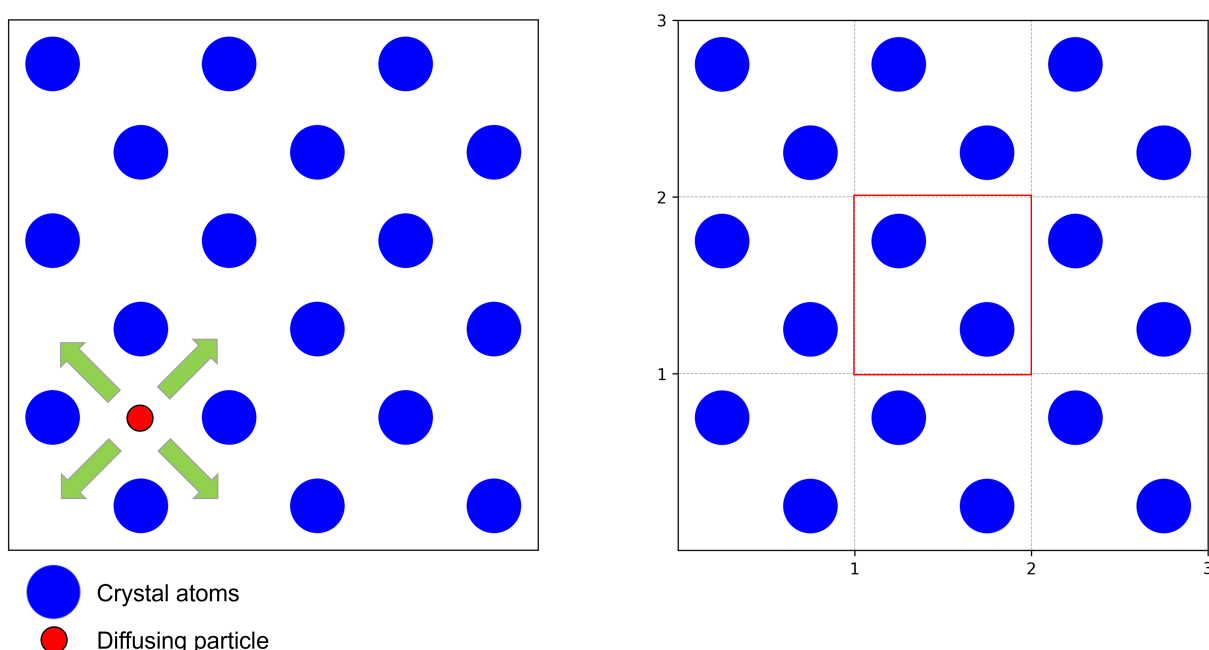


Figure 1: A two-dimensional system consisting of a diffusing particle in a cell-centered tetragonal crystal lattice (left), and the chosen repeating unit used to describe the lattice (outlined in red, right).

In general, the TuTraSt workflow can be split into three parts:

1. *Construction of the potential energy grid:* A potential energy surface (PES) experienced by the diffusing particle is generated on a grid within the chosen repeating unit of the crystal structure. This part of the workflow is not implemented in our software and must be performed using external tools (such as RASPA).
2. *TuTraSt analysis:* The PES is analyzed using the TuTraSt flooding algorithm to identify individual basins accessible to diffusing particles and the transition states between them. From this analysis, several geometric descriptors representative of the system are computed.
3. *Kinetic Monte Carlo (kMC) simulation:* Based on the TuTraSt analysis, a graph network

of basins and possible transitions between them is constructed, and the self-diffusion coefficients of the diffusing particle along the crystal lattice axes are calculated via kMC simulations.

Each of these steps is discussed in detail in the following sections of this chapter.

A.1 Construction of the Potential Energy Grid

The first part of the TuTraSt workflow involves constructing the potential energy surface (PES) experienced by the diffusing particle within the crystal lattice. As noted in the previous section, this part of the workflow is not implemented in our software and must be carried out using external tools, such as RASPA. The procedure for calculating such grids consists of the following steps:

1. *Choosing the repeating unit:* While in some cases simply choosing the unit cell as the repeating unit is sufficient, this is not always true, and careful consideration is required. The choice depends on the nature of the system and the method used for energy calculations. For certain systems—particularly charged systems where quantum chemistry methods such as DFT are employed—if the repeating unit is too small, the calculated energies may depend on its size [4]. In such cases, a sufficiently large supercell must be chosen to eliminate size-dependent effects in the energy calculations.
2. *Constructing a grid on the chosen repeating unit:* Once an appropriate repeating unit is chosen, a three-dimensional grid is overlaid on the structure along the crystal lattice vector directions, serving as a framework for single-point energy calculations. There are several considerations regarding the grid size: a grid that is too coarse may fail to capture the details of the potential energy surface, while a grid that is too fine can lead to high computational cost. It has been found that a voxel size of approximately 0.15–0.2 Å (length of each side) is optimal, although in some cases a size of up to 0.4 Å can be used if appropriate interpolation techniques are applied [4].
3. *Calculating the single particle PES:* Once the grid is constructed, a probe representing the diffusing particle is placed at the center of each voxel, and a single-point energy calculation is performed to determine the energy at that position in the crystal lattice. This procedure is repeated for every voxel in the grid, resulting in the single-particle potential energy surface (PES). The procedure is illustrated in Figure 2. The choice of method for the single-point energy calculations should also depend on the system. For neutral, weakly interacting probes, such as an H_2 molecule, classical force field calculations are usually sufficient. However, more challenging probes—particularly charged ions—require calculations at a higher level of theory, such as density functional theory (DFT) or wavefunction-based quantum methods.

For neutral, weakly interacting probes, single-particle PES provide a sufficient description of the potential energy experienced by particles in a given crystal lattice, as interactions between the diffusing particles are short-ranged. In the case of charged ions, however, the electrostatic

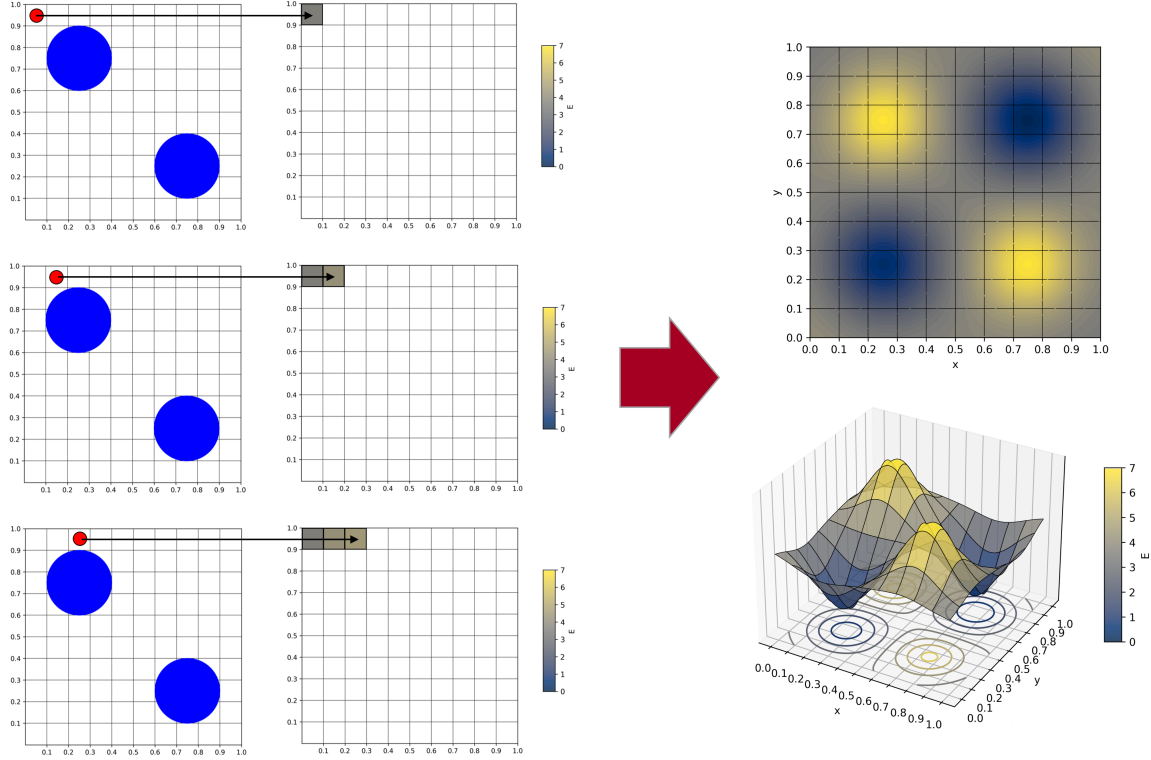


Figure 2: Construction of the single particle potential energy grid.

interactions between particles are long-ranged and influence the effective potential within the lattice. Consequently, single-particle PES are an inadequate description of the system and must be corrected. One approach is the so-called ionic TuTraSt workflow [2], in which a single-particle PES is first constructed, and then a Monte Carlo (MC) simulation is performed on the PES grid populated with the desired number of ions to represent the target loading. From the MC simulation, the probability density of finding ions at various points in space is extracted, and this information is used to calculate an energy correction, which is added to the single-particle PES grid. The result is an effective multi-particle PES grid that accounts for inter-particle interactions.

A.2 TuTraSt analysis

In the second step of the TuTraSt workflow, the PES grid (constructed as described in Section A.1) is partitioned into separate *basins* and *transition states* using the flooding algorithm. Based on this partitioning, a structural analysis of the system is subsequently performed, yielding geometric descriptors of the system.

Before the start of the analysis, the energy levels are discretised, transforming the PES grid into an *energy level matrix*, i.e., a 3D grid in which each voxel contains the energy value at that point in space, floored to the nearest discrete energy level. This process is illustrated in Figure 3. The step between energy levels is a user-defined input. The appropriate value of the energy step varies between systems and has been found to be roughly an order of magnitude smaller than the energy of the first *breakthrough* (i.e., the energy value at which the first percolating channel through the crystal lattice is recorded, see below). Our software

also provides an option to automatically and dynamically adjust the energy step to satisfy this criterion. Additionally, an energy cutoff value is applied to the energy level matrix (also defined by the user): all voxels with an energy level at or above the cutoff value are considered inaccessible (i.e., their energy is set to infinity).

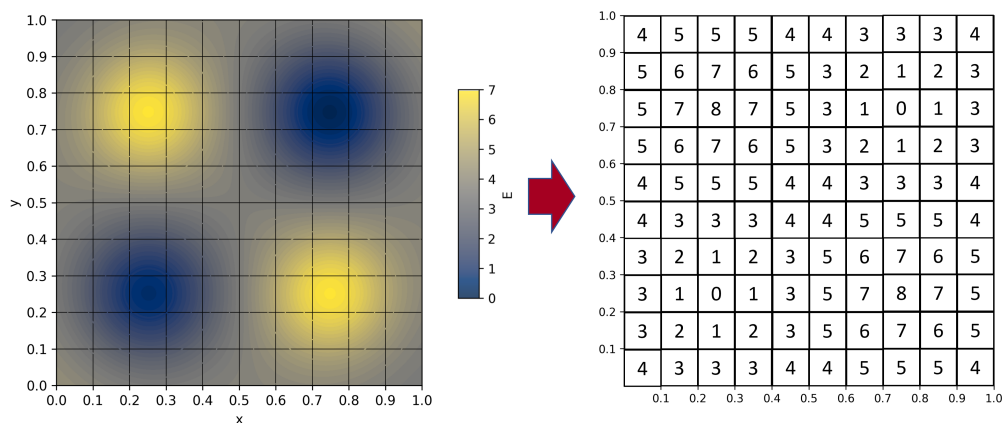


Figure 3: Construction of the energy level matrix. In this case, the energy step is equal to 1 energy unit.

Following the construction of the energy level matrix, the flooding algorithm is applied to partition the space within the PES into individual *basins*—i.e., potential energy wells—and to identify *transition state points*, i.e., points where two adjacent basins merge. The algorithm proceeds by analyzing points in the energy level matrix at each energy level in turn, starting at level 0 and increasing by the energy step until reaching the energy cutoff. At each energy level, the following steps are executed in order:

1. *Initiating new basins:* The energy level matrix is checked for points (or groups of points) at the current energy levels that have no neighbours that are already assigned to a basin (all the neighbours are at a higher energy level). If any such points are found, a new basin is initiated at each such position.
2. *Growing existing basins and identifying transition state points:* After the initiation of new basins, all existing basins are expanded at the current energy level, one layer of points at a time, using the neighbourhood search algorithm. At each basin's border, neighbouring points that do not yet belong to any basin are assigned to the current basin. If a neighbouring point already belongs to a different basin, it is removed from both basin point lists and designated as a transition state point between the two basins. Once basins are connected by at least one transition state point, they are merged into a *basin cluster*, together with all basins connected to the merged basins. If a basin cluster forms a percolating channel through the crystal structure (i.e., forms a continuous path through the repeating unit) in any direction, a *breakthrough* is recorded along that direction at the given energy level. This process is repeated until all points at the current energy level have been processed.

The progression of the flooding algorithm is illustrated in Figure 4.

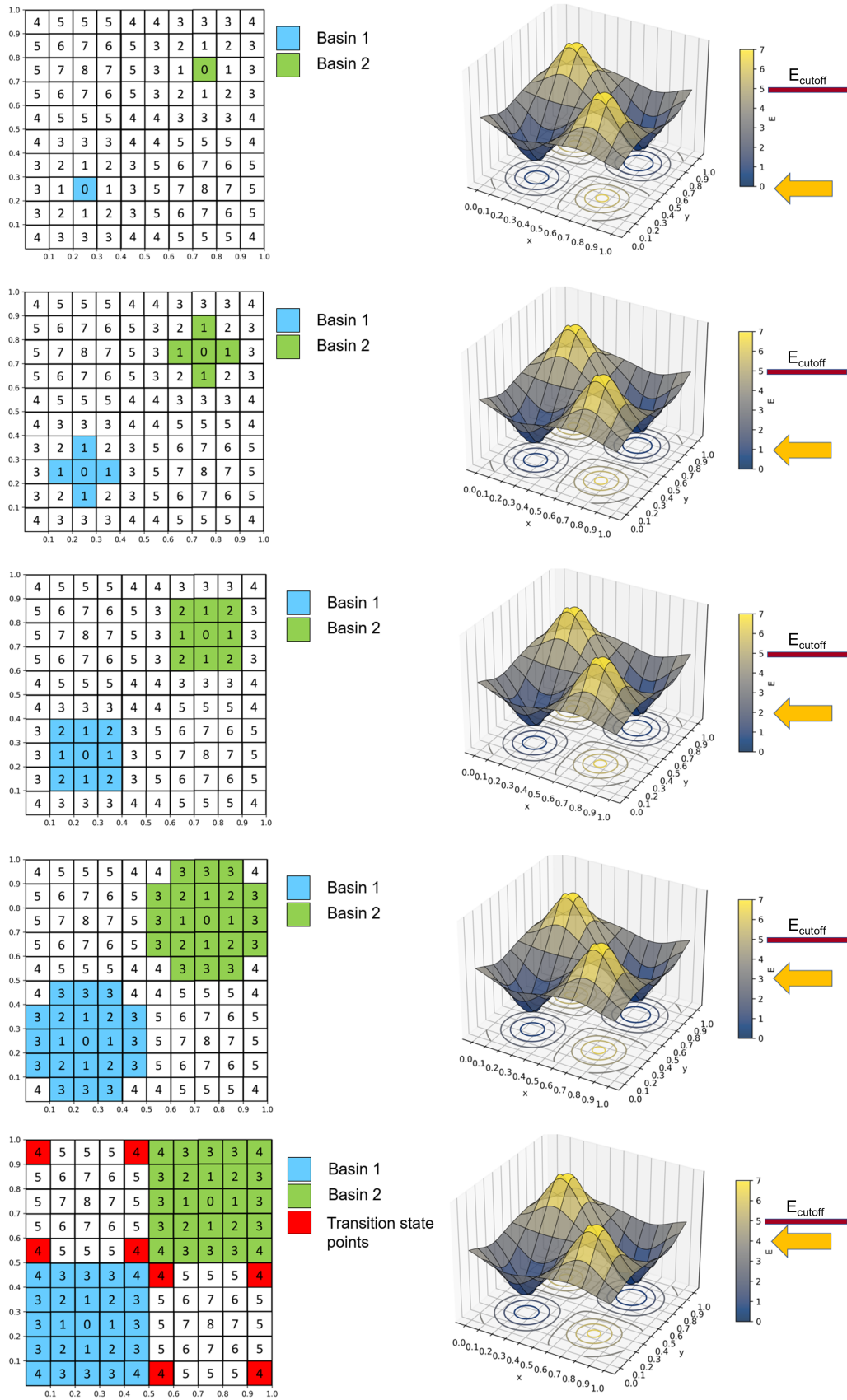


Figure 4: Progression of the flooding algorithm. In this case, the cutoff energy value is equal to 5 energy units.

Once all points at energies below the energy cutoff value have either been assigned to basins or identified as transition state points, the TuTraSt (Tunnel–Transition state) analysis is performed. The first step involves the organisation of *transition states*: all adjacent transition state points between a given pair of basins are merged to form a single transition state corresponding to the transition of a diffusing particle between those two basins.

In the second step, basin clusters within which at least one breakthrough was recorded are identified as *tunnel systems*. Each tunnel system is characterized by its *dimensionality*—the number of linearly independent *directions* of percolating channels within the tunnel system. Each direction is assigned a *breakthrough energy* (the energy at which the breakthrough was first recorded) and a *lowest energy barrier*—the smallest possible maximum energy that a particle must overcome to traverse the tunnel system in that direction. The lowest energy barrier is determined by representing the tunnel system as a graph network, where basins are nodes connected by transition edges with associated energy barriers, and applying a modified minimax algorithm with applied periodic boundary conditions to the graph. Basin clusters in which no breakthroughs were recorded are labeled as *not tunnel-forming*. The tunnel system characterization process is illustrated in Figure 5.

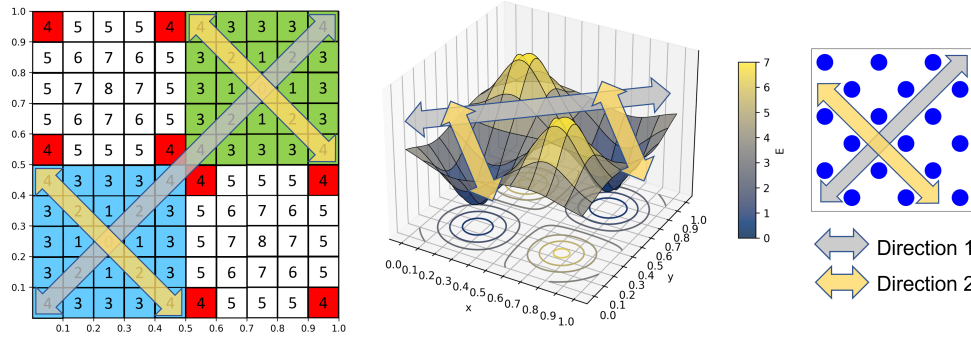


Figure 5: Tunnel system characterization. This example shows a two-dimensional tunnel system, consisting of two basins connected by four transition states, with two breakthrough directions. In both cases, the breakthrough energy and lowest energy barrier are equal to 4 energy units.

Furthermore, additional geometric descriptors, such as the surface area S and volume V , can be readily extracted from the partitioned energy level matrix. The surface area is calculated by summing the areas of all voxel faces that border inaccessible points (i.e., points with energies higher than the cutoff value), as given by Equation 1:

$$S = \sum_{\text{border faces}} \left[(\vec{a} \times \vec{b}) \delta_{\vec{c}} + (\vec{a} \times \vec{c}) \delta_{\vec{b}} + (\vec{b} \times \vec{c}) \delta_{\vec{a}} \right], \quad (1)$$

where $\vec{a}$, $\vec{b}$, and $\vec{c}$ are the side vectors of the voxels forming the grid, and $\delta_{\vec{i}}$ is the *Kronecker delta*, which equals 1 if the face is normal to $\vec{i}$ and 0 otherwise. The volume can be calculated

by summing the volumes of all accessible voxels, as given by Equation 2:

$$V = \sum_{\text{accessible voxels}} \left| \vec{a} \cdot (\vec{b} \times \vec{c}) \right| \quad (2)$$

Both surface area and volume can be reported in the following contexts:

- *Total area/volume*: sum over *all* points with energies below the cutoff value.
- *Accessible area/volume*: sum over points that belong to tunnel systems; points that belong to not tunnel-forming basin clusters are excluded from this metric.
- *Area/volume per tunnel system*: sum only over points belonging to an individual tunnel system.

The surface area and volume calculations are illustrated in Figure 6.

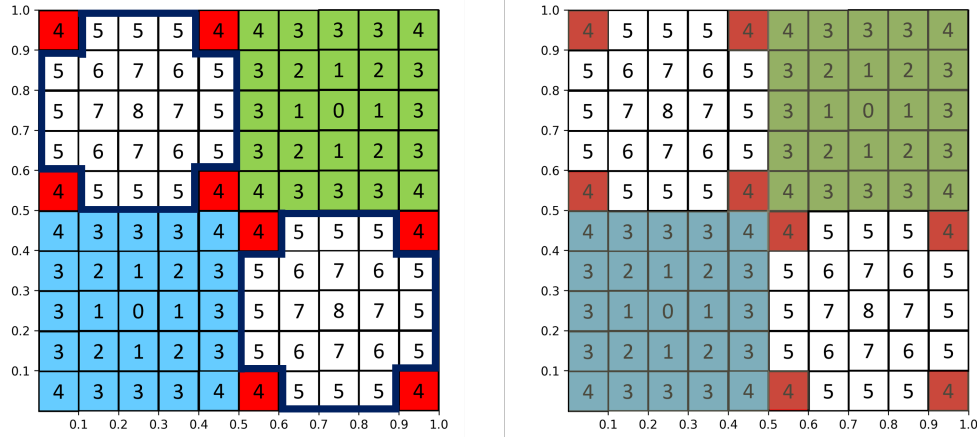


Figure 6: Surface area (dark blue outline, left) and volume (shaded region, right) of the system. In this two-dimensional example, the surface area is equal to 4 units of length, and the volume is equal to 0.58 units of area. The total and accessible values of S and V are identical, as both basins are included in the tunnel system.

A.3 Kinetic Monte Carlo Simulations

The third step of the TuTraSt workflow involves the calculation of self-diffusion coefficients along the crystal lattice axes using kinetic Monte Carlo (kMC) simulations.

First, each tunnel system identified during the TuTraSt analysis (Section A.2) is converted into a three-dimensional graph network. In this representation, individual basins within the tunnel system are treated as nodes, positioned at the spatial coordinates of the lowest-energy point of each basin. The nodes are connected by edges representing *transition processes*.

Each transition process is characterized by a *transition process vector* $\vec{v}$, defined as the vector from the center of the initial basin (basin 1) to the center of the final basin (basin 2), which encodes the displacement length and direction of the diffusing particle, and by a *transition process rate* k , which quantifies the probability that the process occurs per unit time. The

transition process rate for a particle migrating from basin 1 to basin 2 via a given transition state is obtained using the Bennett–Chandler approach [5–7] (Equation 3):

$$k_{1 \rightarrow 2} = \kappa \sqrt{\frac{k_B T}{2\pi m}} P(q^*), \quad (3)$$

where κ is the transmission (symmetry) factor (for an ideal transition state, $\kappa = \frac{1}{2}$), T is the temperature, m is the particle mass, and $P(q^*)$ is the probability of finding the particle at the transition-state coordinates q^* , i.e., at the top of the energy barrier separating basins 1 and 2. This probability is calculated by taking the Boltzmann-weighted sum over all points in the transition state region and normalizing it by the sum over basin 1 and the transition state region (Equation 4):

$$P(q^*) = \frac{\int_{\text{TS}} e^{-E(q^*)/k_B T} dq^*}{\int_{\text{basin 1}} e^{-E(q)/k_B T} dq + \int_{\text{TS}} e^{-E(q^*)/k_B T} dq^*}. \quad (4)$$

For each pair of basins connected by a unique transition state, two transition processes are defined (two per transition state if multiple transition states connect the same basin pair): one from basin 1 to basin 2, with vector $\vec{v}$ and rate $k_{1 \rightarrow 2}$, and one in the reverse direction, with vector $-\vec{v}$ and rate $k_{2 \rightarrow 1}$. The construction of the portion of the graph network corresponding to two basins connected via a transition state is illustrated in Figure 7.

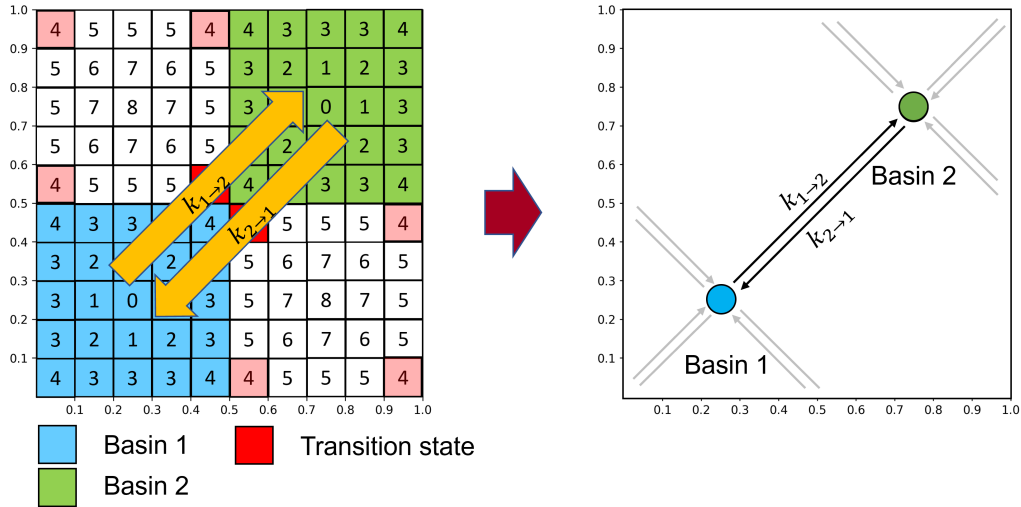


Figure 7: Construction of the portion of the graph network corresponding to basins 1 and 2 connected via the highlighted transition state. The transition process vectors are obtained by taking the difference between the coordinates of the basin centers, and the transition process rates are calculated using Equation 3. To construct the full graph network for this example, the same procedure must be applied to the other three transition states connecting basins 1 and 2.

Once the full graph network for each tunnel system identified within the crystal lattice is constructed, a set of kMC simulations is performed for each tunnel system in order to determine its contribution to the self-diffusion of the diffusing particle along the individual crystal lattice axes. The self-diffusion coefficient is determined using the following procedure:

1. The diffusing particle is placed in a randomly selected basin (i.e., a random node in the

tunnel-system graph). Time is set to $t = 0$, and the particle's initial position is set to $\vec{r} = (0, 0, 0)$.

2. From all available transition processes originating in the basin currently occupied by the particle (i.e., all outgoing edges from the current node), one process is selected at random. The selection probability is weighted by the corresponding transition process rate k_i and normalized by the sum of all outgoing rates, as given by Equation 5:

$$P_i = \frac{k_i}{\sum_{\text{basin}} k_i}, \quad (5)$$

where P_i is the probability of selecting the i -th process and the denominator represents the sum over all transition process rates starting from the current basin. After selecting the process, the particle is moved from the initial basin to the target basin. The simulation time is advanced according to the Gillespie algorithm [8] (Equation 6):

$$\Delta t = -\frac{\ln(\rho)}{\sum_{\text{basin}} k_i}, \quad (6)$$

where ρ is a random number uniformly distributed between 0 and 1. The particle position is updated by adding the displacement vector of the chosen transition process. This step is repeated N times (where N is a user-defined parameter), yielding a particle trajectory $\vec{r}(t)$, as illustrated in Figure 8.

3. From the particle trajectory, mean square displacement (MSD) curves are computed along the crystal lattice directions:

$$\text{MSD}_i = \left\langle (r_i(t) - r_i(t_0))^2 \right\rangle, \quad (7)$$

where MSD_i is the mean square displacement along direction i , and $r_i(t)$ and $r_i(t_0)$ are the particle coordinates along that direction at the time t and the chosen initial time t_0 , respectively.

The diffusion coefficient along direction i in a tunnel system T is obtained from the slope of the MSD curve:

$$D_i^T = \frac{1}{2} \lim_{t \rightarrow \infty} \frac{d}{dt} \left\langle (r_i(t) - r_i(t_0))^2 \right\rangle. \quad (8)$$

To ensure that the diffusive regime is reached, the slope is evaluated only over the portion of the trajectory where the particle has traveled between one and two unit-cell lengths along the respective direction. If the particle fails to traverse at least two unit-cell lengths within the simulation time, the diffusion in that direction is considered negligible and the corresponding diffusion coefficient is set to zero.

For each tunnel system, this procedure is repeated multiple times, and the diffusion coefficient is taken as the average over all runs.

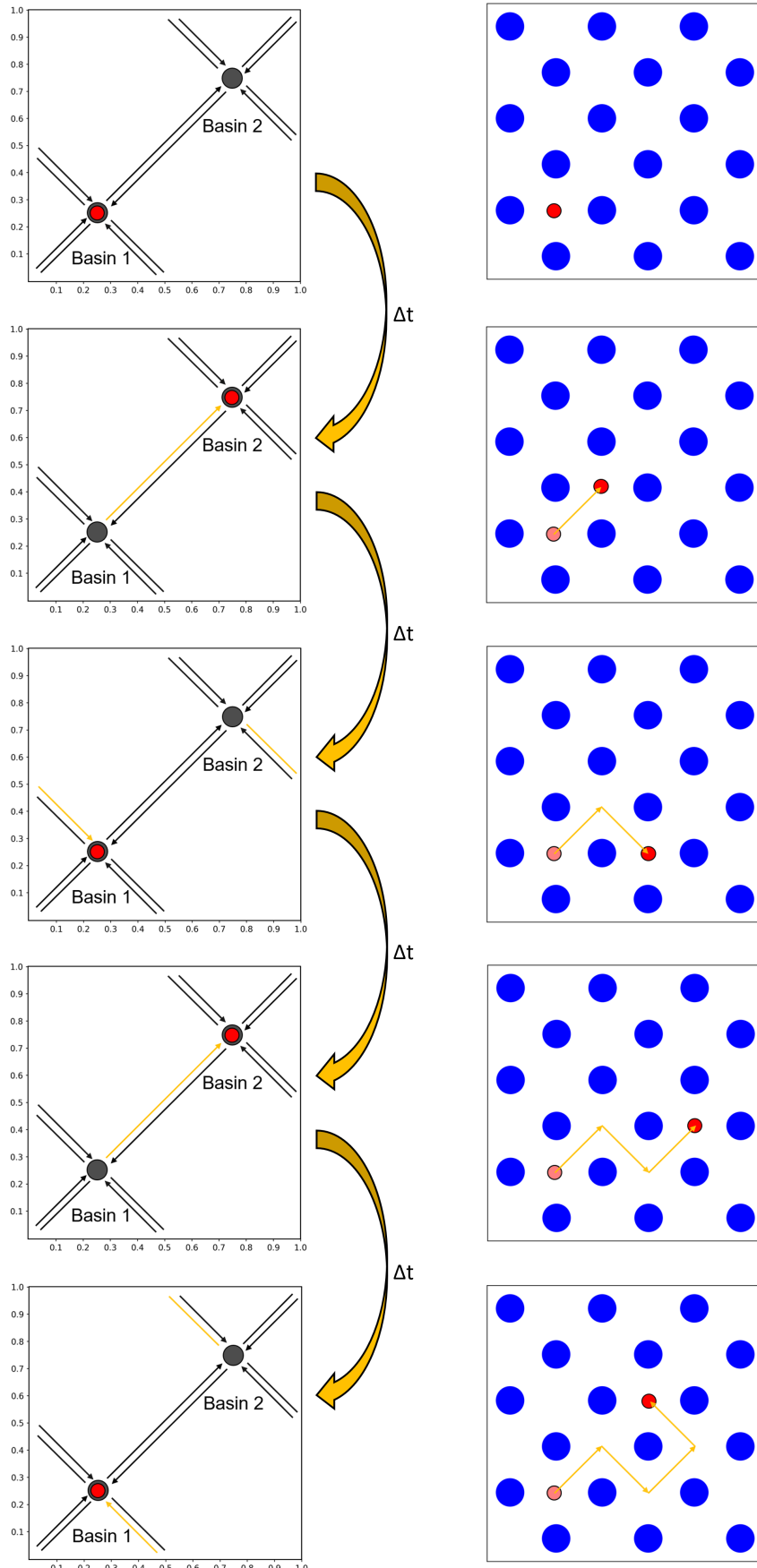


Figure 8: Progression of the kMC simulation. The right-hand side of the figure shows the trajectory of the diffusing particle, $\vec{r}(t)$.

To obtain the total diffusion coefficients along the crystal lattice direction i , a weighted average over all tunnel systems is computed:

$$D_i = \sum_T P(T) D_i^T, \quad (9)$$

where the weight $P(T)$ is the probability of finding the particle within tunnel system T . It is calculated as the Boltzmann-weighted sum over all voxels belonging to that tunnel system, normalized by the Boltzmann-weighted sum over the total accessible volume of the system (i.e., all voxels with energy below the cutoff):

$$P(T) = \frac{\int_T e^{-E(q)/k_B T} dq}{\int_V e^{-E(q)/k_B T} dq}. \quad (10)$$